Disease Diagnosis Bot

Transforming healthcare accessibility through intelligent, symptom-based diagnosis powered by artificial intelligence

Code: The Health Bot Implementation

```
def start_health_bot():
    health_data = {
        "Migraine": ["headache", "nausea", "light sensitivity"],
        "Heatstroke": ["high fever", "confusion", "no sweating", "rapid pulse"],
        "Allergy": ["itching", "sneezing", "watery eyes", "skin rash"],
        "Common Cold": ["sneezing", "sore throat", "runny nose", "cough"],
        "COVID-19": ["fever", "dry cough", "loss of taste", "shortness of breath"],
        "Food Poisoning": ["vomiting", "diarrhea", "stomach cramps", "nausea"],
        "Malaria": ["high fever", "shivering", "sweating", "headache"],
        "Flu": ["fever", "chills", "muscle ache", "fatigue", "cough"]
    }

    advice_list = {
        "Migraine": "Rest in a dark room and avoid bright screens.",
        "Heatstroke": "Move to a cool place and seek emergency medical help.",
        "Allergy": "Take an antihistamine and find out what triggered it.",
        "Common Cold": "Drink warm fluids and get some sleep.",
        "COVID-19": "Isolate yourself and check your oxygen levels.",
        "Food Poisoning": "Drink ORS and stick to light food like curd rice.",
        "Malaria": "You need a blood test. Visit a clinic for anti-malarials.",
        "Flu": "Take paracetamol for fever and stay hydrated."
    }

    p_name = input("Patient Name: ")
    p_age = input("Patient Age: ")
    current_symptoms = []

    while True:
        user_stuff = input("\nWhat symptoms are you feeling? (separate by commas): ").lower()
        new_list = [item.strip() for item in user_stuff.split(',')]

        for s in new_list:
            if s not in current_symptoms:
                current_symptoms.append(s)

        print(f"\nChecking record for {p_name}...")
        found_matches = []

        for disease, symp_list in health_data.items():
            common = set(current_symptoms).intersection(set(symp_list))
            if common:
                prob = (len(common) / len(symp_list)) * 100
                found_matches.append((disease, prob))

        if not found_matches:
            print("Sorry, no match found in my database yet.")
        else:
            found_matches.sort(key=lambda x: x[1], reverse=True)
            best_guess, final_score = found_matches[0]

            print(f">>> Result for {p_name} (Age {p_age}):")
            print(f">>> Likely: {best_guess} | Confidence: {final_score:.1f}%")
            print(f">>> Advice: {advice_list[best_guess]}")

        check_more = input("\nAny other symptoms to add? (y/n): ").lower()
        if check_more != 'y':
            happy = input("Is this diagnosis helpful? (y/n): ").lower()
            if happy == 'y':
                print(f"Get well soon!")
                break
```

How Bot Works: The Core Logic

01

Patient Profile Collection

The bot starts by collecting patient name and age using input() functions, creating a personalized health record.

02

Symptom Input & Cleaning

Users enter symptoms separated by commas. The code converts to lowercase and strips whitespace, then adds unique symptoms to the current_symptoms list.

03

Disease Matching Algorithm

The bot compares user symptoms against the health_data dictionary using set intersection. It calculates confidence percentage: (matching symptoms / total disease symptoms) × 100.

04

Result & Advice Delivery

The bot sorts matches by confidence score, displays the highest probability disease, and provides specific medical advice from the advice_list dictionary.

Data Structure

health_data Dictionary

Contains 8 diseases (Migraine, Heatstroke, Allergy, Common Cold, COVID-19, Food Poisoning, Malaria, Flu) with their associated symptoms as lists. This is the knowledge base your bot uses for diagnosis.

advice_list Dictionary

Paired with health_data, contains personalized medical advice for each disease. When a disease is diagnosed, the corresponding advice is displayed to the user.

current_symptoms List

Dynamically built during the conversation. Stores all unique symptoms the patient reports, preventing duplicates and building a comprehensive symptom profile for accurate matching.

The Matching Algorithm: How Diagnosis Works

01

Set Intersection

For each disease, the code uses `set(current_symptoms).intersection(set(symp_list))` to find common symptoms between what the patient reported and what the disease requires.

02

Confidence Calculation

Probability = (number of matching symptoms / total symptoms for that disease) × 100. This gives a percentage score showing how likely the diagnosis is.

03

Match Collection

All diseases with at least one matching symptom are added to `found_matches` list as tuples: `(disease_name, confidence_percentage)`.

04

Ranking & Selection

The list is sorted by confidence score in descending order. The disease with the highest confidence becomes the primary diagnosis recommendation.

Unique Additions: Migraine & Heatstroke

These two diseases make the project special because they broaden the bot beyond common diagnosis patterns and add more practical real-world coverage.

Migraine

Symptoms: headache, nausea, light sensitivity

Advice: Rest in a dark room and avoid bright screens.

This is a common neurological condition that many diagnosis systems overlook.

Heatstroke

Symptoms: high fever, confusion, no sweating, rapid pulse

Advice: Move to a cool place and seek emergency medical help.

This is a critical heat-related emergency condition, especially relevant in India's climate.

These additions demonstrate your understanding of real-world health conditions and make your bot more practical for actual use cases.

User Interaction Flow: The While Loop

01

Input Phase

User enters symptoms separated by commas. Code converts to lowercase and strips whitespace for consistency.

02

Accumulation

New symptoms are added to current_symptoms list only if they're not already present, which prevents duplicates.

03

Analysis

Bot runs the matching algorithm against all 8 diseases and displays the top result with confidence percentage and advice.

04

Loop Control

User is asked “Any other symptoms to add? (y/n)”. If yes, the loop continues. If no, user is asked “Is this diagnosis helpful?” to confirm satisfaction or request more details.

Real-World Use Cases for Your Bot

First-Level Triage

Patients can quickly input symptoms and get preliminary guidance before visiting a doctor, helping them understand urgency levels

Health Education

The bot teaches users about disease symptoms and appropriate responses. The advice_list provides actionable next steps for each condition

Symptom Documentation

By collecting patient name, age, and symptoms, the bot creates a health record that can be shared with healthcare professionals for more informed consultations

Potential Enhancements to Your Bot

Expand Disease Database

Add more diseases and symptoms to improve diagnostic accuracy and coverage

Severity Scoring

Implement a severity level (mild, moderate, severe) based on symptom combinations to better guide users on urgency

Symptom Weighting

Give different symptoms different importance levels. For example, "high fever" might be weighted more heavily than "sneezing"

User Feedback Loop

Track which diagnoses users found helpful vs unhelpful to continuously improve the matching algorithm's accuracy

Key Strengths of Your Implementation

Clean Data Handling

The code properly cleans user input (lowercase, strip whitespace) ensuring consistent matching

Duplicate Prevention

The bot checks if symptoms already exist before adding them, maintaining a clean symptom list

Confidence Scoring

Uses mathematical calculation (intersection / total) to provide quantifiable confidence percentages rather than vague results

User-Centric Design

Collects patient name and age, provides personalized output, and includes a feedback loop to verify user satisfaction

Conclusion: Health Bot Achievement

A functional disease diagnosis bot that demonstrates solid Python fundamentals including:

- Dictionary and list data structures
- Set operations for efficient matching
- Mathematical calculations for confidence scoring
- User input handling and validation
- Control flow with while loops and conditionals
- String manipulation (lowercase, strip)

This project shows practical application of programming concepts to solve real-world healthcare accessibility challenges. The bot can help users understand their symptoms and make informed decisions about seeking medical care.